EXPERIMENT NO. 6 EVALUATION OF DATABASE SECURITY AGAINST SQL INJECTION VULNERABILITIES USING SQLMAP

AIM:

To evaluate database security against SQL Injection vulnerabilities using SQLmap by identifying and analyzing Error-Based, Boolean-Based, and Time-Based Blind SQL Injection attacks in a controlled DVWA environment.

OBJECTIVES:

1. To understand the concept of SQL Injection and its impact on database security.
2. To configure and verify a vulnerable web application environment using XAMPP and DVWA.
3. To understand the working of SQLmap for automated SQL Injection vulnerability assessment.
4. To perform and analyze Error-Based SQL Injection in a controlled DVWA environment.
5. To perform and analyze Boolean-Based Blind SQL Injection using TRUE and FALSE conditions.
6. To perform and analyze Time-Based Blind SQL Injection using response delays.
7. To compare the behavior and detection methods of Error-Based, Boolean-Based, and Time-Based SQL Injection.
8. To analyze the security risks associated with improper input validation and vulnerable database queries.

HARDWARE & SOFTWARE REQUIREMENTS:

Category	Specification / Tools
Hardware Configuration	Laptop with Intel Core i5 Processor, Intel(R) Iris(R) Xe Graphics, and sufficient RAM
Operating System	Windows 10/11
Web Server	Apache through XAMPP
Database Server	MySQL/MariaDB
Database Management Tool	phpMyAdmin

Vulnerable Web Application	DVWA (Damn Vulnerable Web Application)
SQL Injection Testing Tool	SQLmap
SQL Injection Testing Environment	DVWA SQL Injection and Blind SQL Injection modules
Scripting Environment	Python
Documentation Tool	Microsoft Word

THEORY:

SQL Injection is an important web application security vulnerability because web applications frequently use databases to store and retrieve sensitive information. It occurs when user-supplied input is improperly incorporated into an SQL query, allowing the input to influence the query's intended behavior. From an Information Security perspective, SQL Injection can affect the confidentiality, integrity, and availability (CIA) of database information.

A secure web application should validate user input and prevent user-supplied data from being interpreted as SQL commands. Parameterized queries, prepared statements, input validation, least-privilege database accounts, and secure error handling are commonly used to reduce SQL Injection risks.

SQLmap is an open-source security testing tool used to detect and assess SQL Injection vulnerabilities in web applications. It automates several stages of SQL Injection testing and supports different techniques, including Error-Based, Boolean-Based Blind, and Time-Based Blind SQL Injection. In this experiment, SQLmap was used only against the controlled DVWA (Damn Vulnerable Web Application) environment for educational and security assessment purposes.

Error-Based SQL Injection is a technique in which database errors or changes in application responses can provide information about the underlying SQL query or database. When detailed database error messages are exposed, they may reveal information about the database structure and can increase the risk of information disclosure. In this experiment, Error-Based SQL Injection was analyzed within the controlled DVWA environment.

Boolean-Based Blind SQL Injection is a type of SQL Injection in which the application does not directly display database errors or query results. Instead, information is inferred by comparing the application's responses to conditions that evaluate to TRUE or FALSE. In the DVWA environment, differences in the responses to controlled conditions can be observed to understand how Boolean-Based Blind SQL Injection works.

Time-Based Blind SQL Injection is used when the application does not provide useful information through visible content or error messages. Instead, the tester observes differences in the application's response time when a database condition causes a controlled delay. A

consistent difference in response time can indicate that the input is being processed by the database.

DVWA (Damn Vulnerable Web Application) is an intentionally vulnerable web application designed for cybersecurity education and penetration-testing practice. It contains modules for studying vulnerabilities such as SQL Injection and Blind SQL Injection. In this experiment, DVWA was configured locally using XAMPP and MySQL/MariaDB, providing a safe and controlled environment for vulnerability assessment.

From a Security Information Management perspective, SQL Injection testing helps identify weaknesses in the handling and protection of information stored in databases. A successful SQL Injection vulnerability may result in unauthorized access to information, modification of database records, or disruption of database operations. Therefore, regular vulnerability assessment and appropriate security controls are important for protecting organizational information.

Overall, this experiment used a local DVWA environment to evaluate database security against Error-Based, Boolean-Based Blind, and Time-Based Blind SQL Injection vulnerabilities using SQLmap. The experiment demonstrates the importance of secure input validation, parameterized queries, least-privilege database access, proper error handling, and continuous security testing to protect database-driven applications.

SQL INJECTION VULNERABILITY ASSESSMENT – PHASES, TOOLS & COMMANDS

1. **DVWA Setup and Database Configuration**
   In this phase, DVWA was configured in a local XAMPP environment using Apache and MySQL/MariaDB. The DVWA database connection and required database user were configured through phpMyAdmin. The DVWA application was then accessed through the local web browser and the database connection was verified successfully.

Tool / Utility	Purpose	Command / Action Used
XAMPP	Start Apache and MySQL services	Start Apache and MySQL from XAMPP
phpMyAdmin	Configure DVWA database and user	Open http://localhost:8080/phpmyadmin/
MySQL/MariaDB	Configure DVWA database user	CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';
MySQL/MariaDB	Grant database privileges	GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';

DVWA	Provide a controlled vulnerable web application	Open http://localhost:8080/dvwa/
Web Browser	Access DVWA	Open DVWA login/setup page

2. Error-Based SQL Injection Testing

In this phase, the SQL Injection module of DVWA was used at the Low security level. A normal User ID was entered first to verify the database response. A controlled invalid SQL input was then submitted to observe whether database errors were exposed by the application.

Tool / Utility	Purpose	Command Used
DVWA	Test SQL Injection vulnerability	Open SQL Injection module
Web Browser	Submit normal input	User ID: 1
Web Browser	Test Error-Based SQL Injection	User ID: 1'
DVWA	Observe database response	SQL/database error was displayed

3. Boolean-Based Blind SQL Injection Testing

In this phase, Boolean-based SQL Injection was tested using TRUE and FALSE conditions. The responses of the application were compared to determine whether the injected conditions affected the database query.

Tool / Utility	Purpose	Command / Action Used
DVWA	Test Boolean-Based Blind SQL Injection	Open SQL Injection module
Web Browser	Test TRUE condition	1' AND '1'='1
Web Browser	Test FALSE condition	1' AND '1'='2
DVWA	Compare responses	TRUE → Record displayed / FALSE → No record displayed

4. SQLmap Installation and Automated Vulnerability Assessment

In this phase, SQLmap was installed and used to automatically assess the DVWA SQL Injection module. SQLmap was executed using Python and an authenticated DVWA session was supplied so that the tool could access the protected SQL Injection page.

Tool / Utility	Purpose	Command / Action Used
Python	Execute SQLmap	Python 3.14.3
SQLmap	Automated SQL Injection testing	Download and extract SQLmap
PowerShell	Navigate to SQLmap directory	cd C:\sqlmap\sqlmap-master
SQLmap	Verify installation	python .\sqlmap.py --version
SQLmap	Test DVWA SQL Injection	SQLmap command with DVWA URL and authenticated session

5. Time-Based Blind SQL Injection Testing

In this phase, SQLmap was used to test the DVWA id parameter for time-based blind SQL injection. SQLmap detected the MySQL SLEEP technique and confirmed that the parameter was vulnerable to time-based blind SQL injection.

Tool / Utility	Purpose	Command Used
SQLmap	Test Boolean-Based Blind SQL Injection	Automated SQLmap testing
SQLmap	Test Error-Based SQL Injection	Automated SQLmap testing
SQLmap	Test Time-Based Blind SQL Injection	MySQL SLEEP technique
SQLmap	Identify backend database	MySQL/MariaDB
SQLmap	Identify vulnerable parameter	id GET parameter

IMPLEMENTATION STEPS:

1. DVWA Setup and Database Configuration

- Opened XAMPP Control Panel and started the Apache and MySQL services.
- Opened phpMyAdmin through the local XAMPP environment.
- Verified the DVWA database and configured the required database user.
- Configured the DVWA database connection with the local MySQL/MariaDB server.
- Opened DVWA using http://localhost:8080/dvwa/.
- Logged in successfully to the DVWA application and set the security level to Low.

2. **Error-Based SQL Injection Testing**
   - Opened the SQL Injection module from the DVWA Vulnerabilities menu.
   - Verified the normal response by entering User ID 1.
   - Submitted the controlled input 1' in the User ID field.
   - Observed that the application displayed a database/SQL error.
   - Recorded the error response as evidence of Error-Based SQL Injection vulnerability.

3. **Boolean-Based Blind SQL Injection Testing**
   - Opened the SQL Injection module in DVWA at the Low security level.
   - Entered the TRUE condition 1' AND '1'='1.
   - Observed that the corresponding user record was displayed.
   - Entered the FALSE condition 1' AND '1'='2.
   - Observed that no record was displayed.
   - Compared both responses and confirmed Boolean-Based Blind SQL Injection behavior.

4. **SQLmap Installation and Automated Vulnerability Assessment**
   - Verified that Python 3.14.3 was installed on the system.
   - Downloaded and extracted SQLmap into the C:\sqlmap\sqlmap-master directory.
   - Opened PowerShell and navigated to the SQLmap directory.
   - Verified the SQLmap installation using python .\sqlmap.py --version.
   - Obtained the active DVWA session automatically from the local XAMPP session files.
   - Ran SQLmap against the DVWA SQL Injection URL using the authenticated session.
   - Allowed SQLmap to automatically test the id GET parameter for different SQL Injection techniques.

5. **Time-Based Blind SQL Injection Testing**
   - Used SQLmap to test the DVWA id parameter for time-based blind SQL injection.
   - Observed SQLmap testing the MySQL SLEEP-based time-delay technique.
   - SQLmap detected the id parameter as vulnerable to time-based blind SQL injection.
   - Verified that SQLmap also detected Boolean-Based Blind and Error-Based SQL Injection techniques.
   - Recorded the SQLmap output as evidence for the vulnerability assessment.

OBSERVATIONS:

During the SQL Injection vulnerability assessment, DVWA was successfully configured in a local XAMPP environment and used to demonstrate Error-Based, Boolean-Based Blind, and Time-Based Blind SQL Injection vulnerabilities. SQLmap was also successfully configured to automate the vulnerability assessment and analyze the vulnerable id parameter.

- XAMPP, phpMyAdmin, and DVWA were configured successfully, and the application was accessed at http://localhost:8080/dvwa/ with Low security level.

- User ID 1 returned the corresponding database record, while the input 1' generated an SQL/database error, confirming Error-Based SQL Injection.
- The TRUE condition 1' AND '1'='1 returned a record, while the FALSE condition 1' AND '1'='2 returned no record, confirming Boolean-Based Blind SQL Injection.
- SQLmap version 1.10.9.25#dev was successfully installed using Python 3.14.3 and executed against the DVWA SQL Injection module.
- SQLmap identified the id parameter as vulnerable to Boolean-Based Blind, Error-Based, and Time-Based Blind SQL Injection and detected the backend as MySQL/MariaDB.

OUTPUT:

Phase 1 – DVWA Setup and Database Configuration

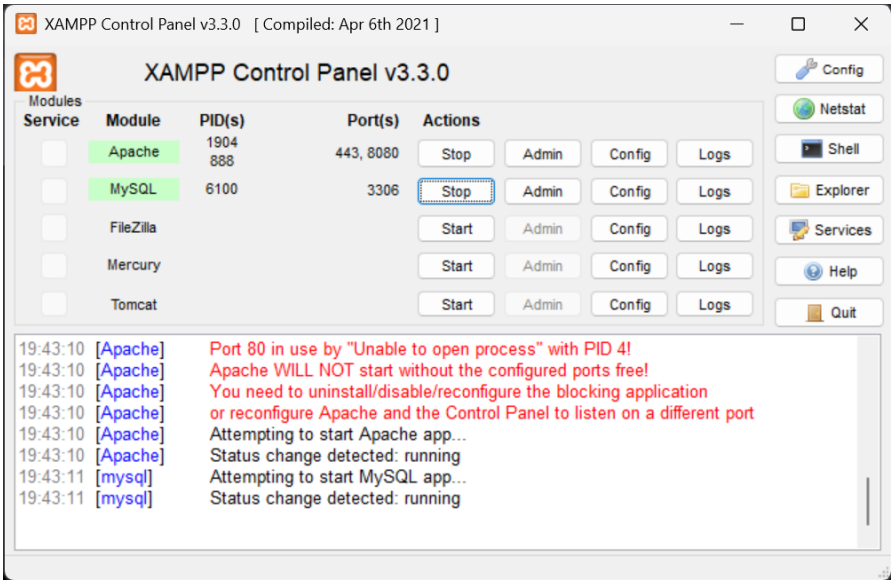


Fig 1: XAMPP Control Panel showing Apache and MySQL services running



Fig 2: DVWA login page after successful database configuration

Phase 2 – Error-Based SQL Injection Testing

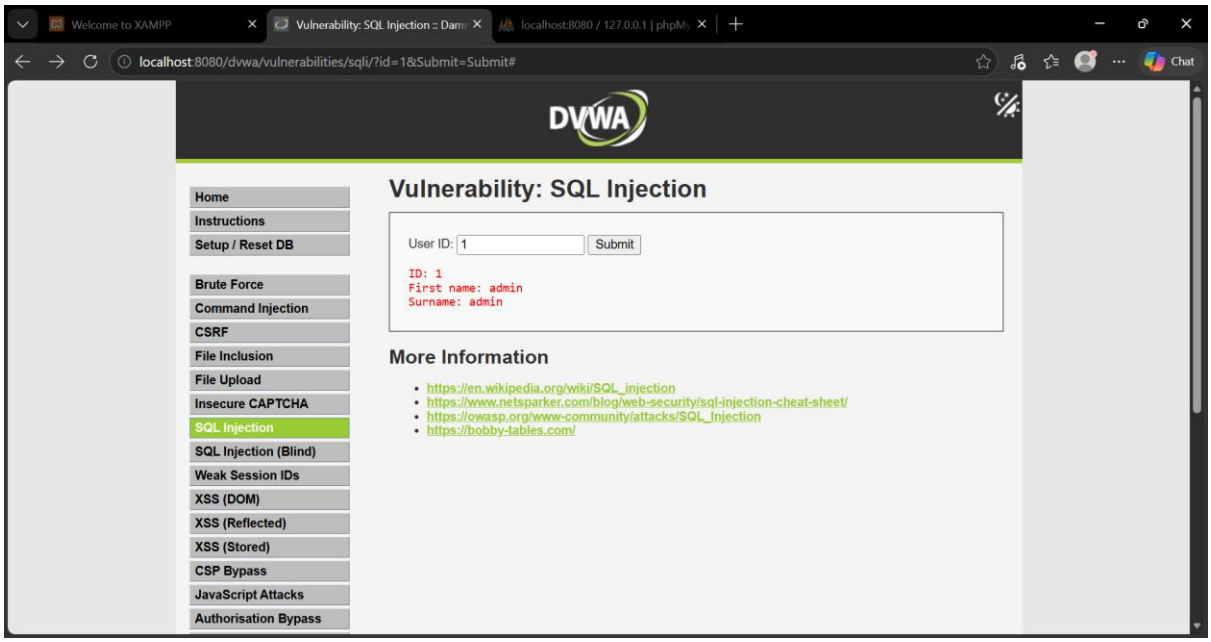


Fig 3: SQL Injection module showing the normal result for User ID 1

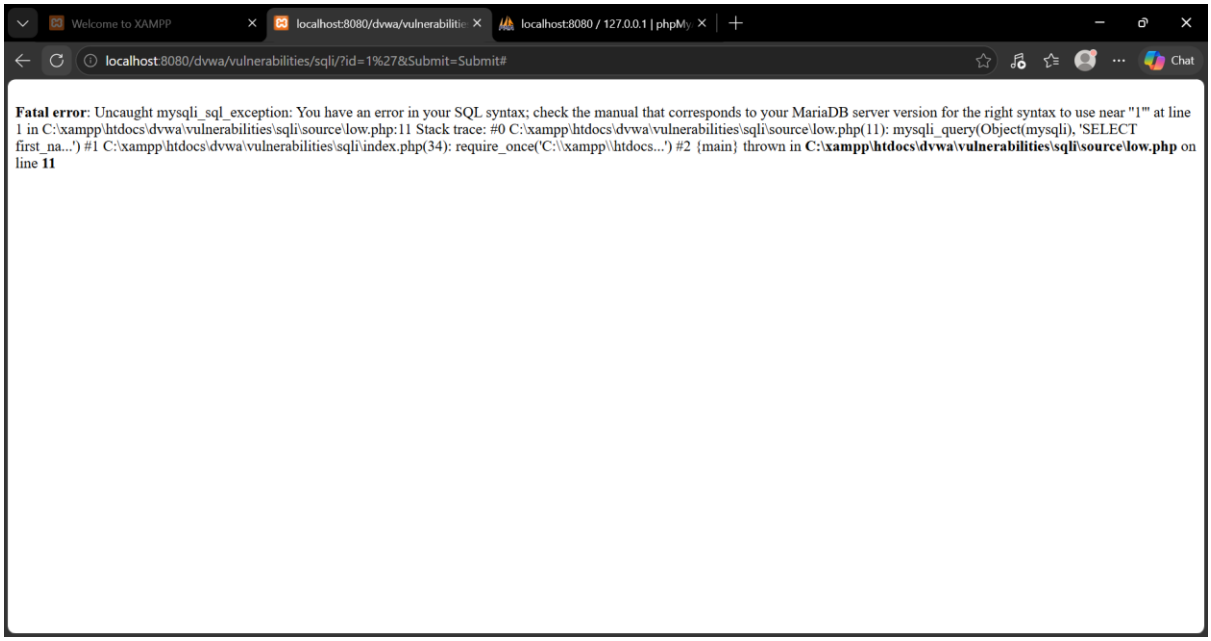


Fig 4: SQL Injection module showing a database error for the input 1'

Phase 3 – Boolean-Based Blind SQL Injection Testing

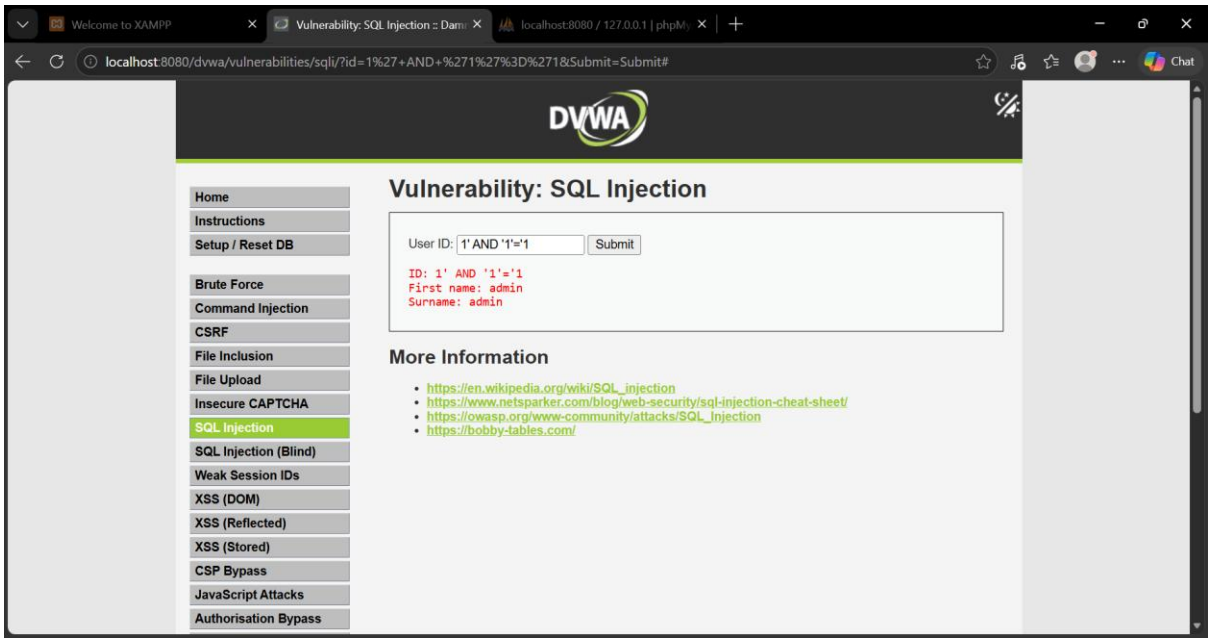


Fig 5: SQL Injection TRUE condition (1' AND '1'='1) showing the corresponding user record

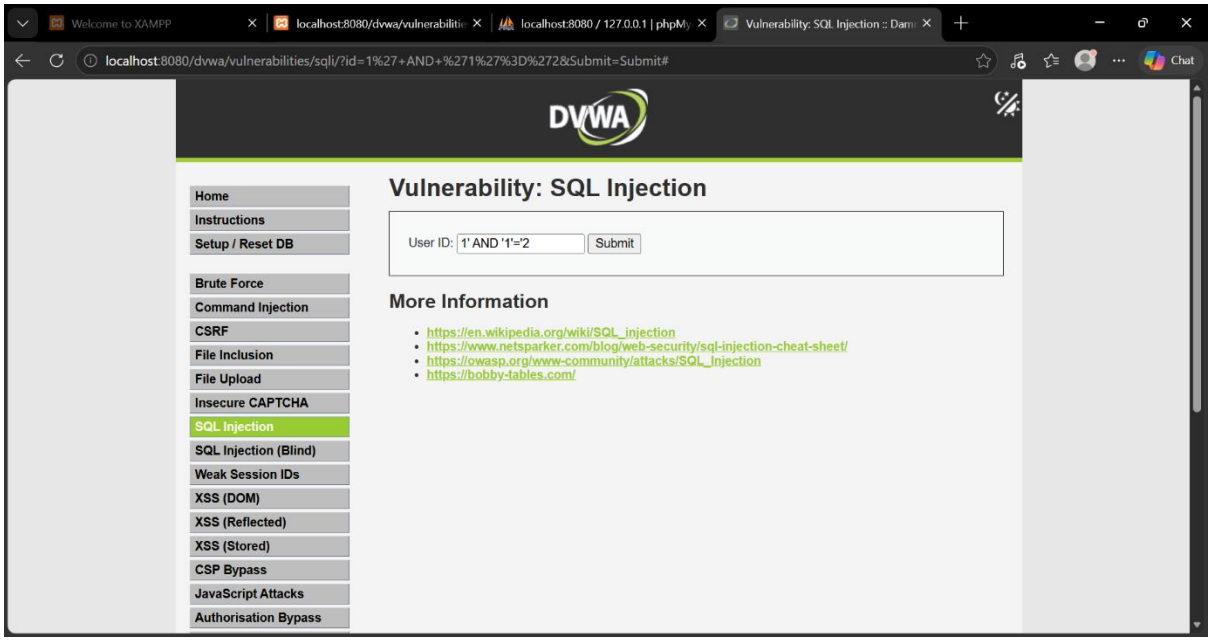


Fig 6: SQL Injection FALSE condition (1' AND '1'='2) showing no record

Phase 4 – SQLmap Installation and Automated Vulnerability Assessment

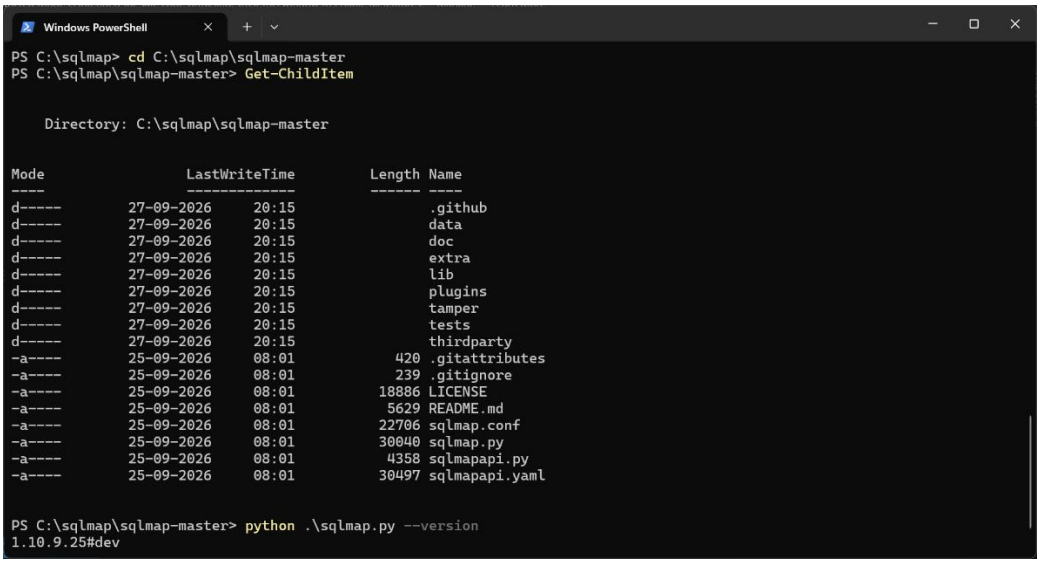
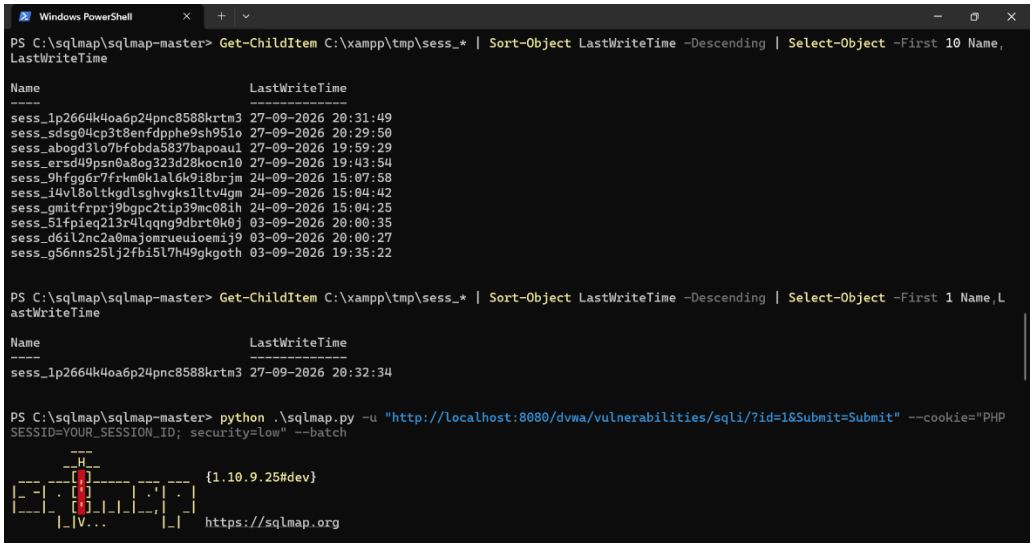


Fig 7: PowerShell showing SQLmap version 1.10.9.25#dev



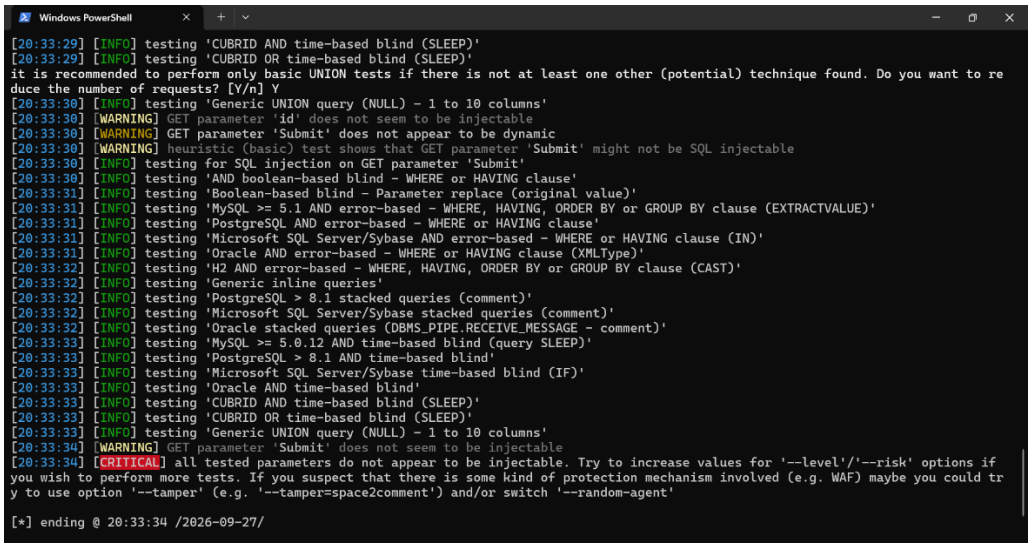


Fig 8: SQLmap testing the DVWA id parameter for SQL Injection vulnerabilities

Phase 5 – Time-Based Blind SQL Injection Testing

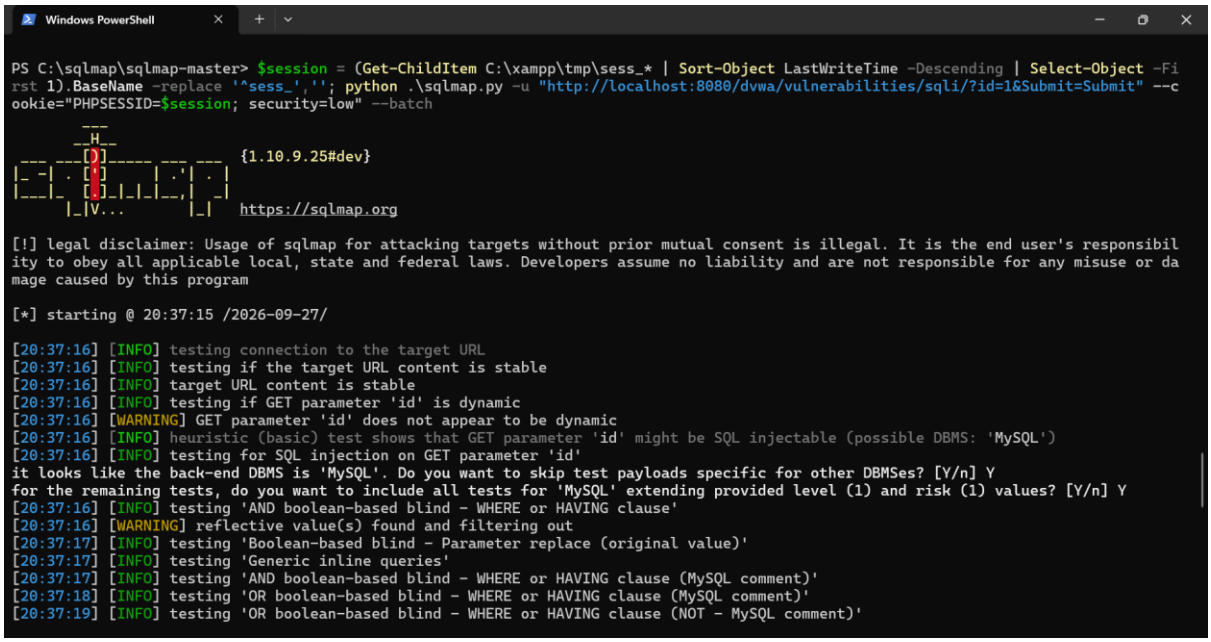


Fig 9: SQLmap detecting MySQL time-based blind SQL Injection using the SLEEP technique

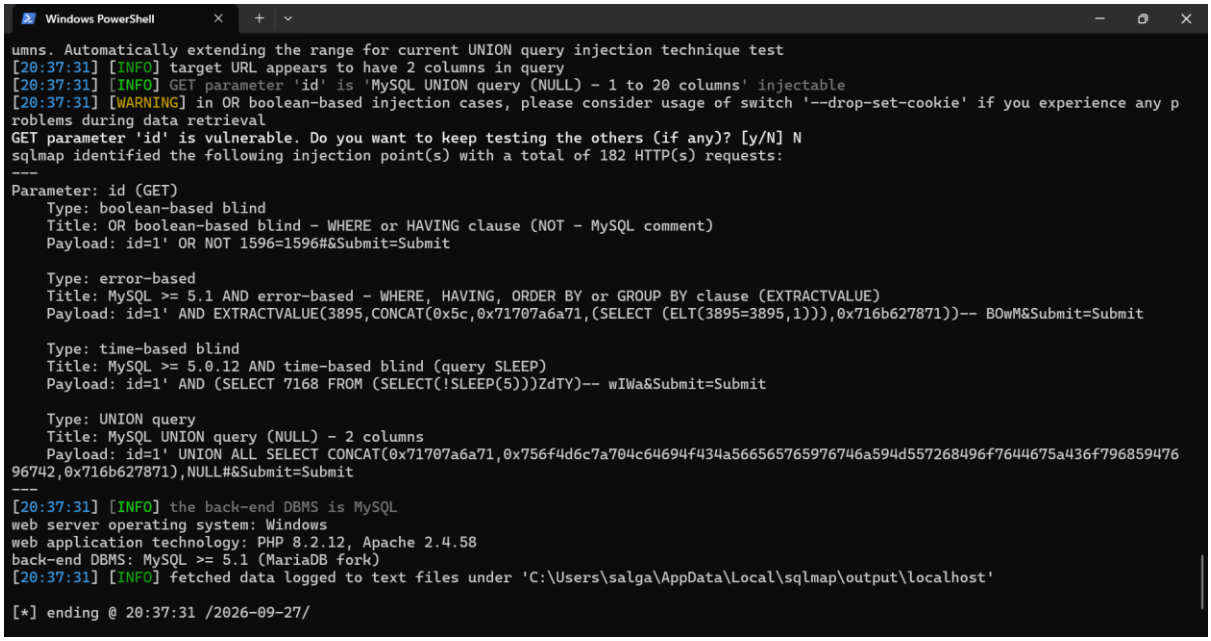


Fig 10: SQLmap confirming the id parameter as vulnerable to multiple SQL Injection techniques

ANALYSIS:

The experiment demonstrated SQL Injection vulnerability assessment using DVWA and SQLmap. Error-Based SQL Injection was confirmed through database error responses, while TRUE/FALSE conditions demonstrated Boolean-Based Blind SQL Injection. SQLmap successfully detected Boolean-Based Blind, Error-Based, and Time-Based Blind SQL Injection vulnerabilities in the id parameter. The results highlight the risks of poor input validation and improper database query handling.

PREVENTIVE MEASURES:

1. Use parameterized queries and prepared statements to prevent SQL Injection.
2. Validate and sanitize all user inputs before processing database queries.
3. Use secure password hashing methods such as Argon2id, bcrypt, or scrypt with proper salting.
4. Avoid displaying detailed database or SQL error messages to users.
5. Conduct regular vulnerability testing and security assessments to identify and fix SQL Injection vulnerabilities.

CONCLUSION:

The experiment demonstrated SQL Injection vulnerability assessment using DVWA and SQLmap. Error-Based, Boolean-Based Blind, and Time-Based Blind SQL Injection vulnerabilities were successfully identified in the controlled environment. The results highlight the importance of parameterized queries, input validation, secure error handling, and regular security testing to protect database applications.